

Improving ArUco Tag Detection Using Deep Learning

Motivation

EPFL Xplore is a student-led swiss robotics association, part of the MAKE Initiative at EPFL. It builds a Martian-like Rover each year to compete in the European Rover Challenge (ERC) in Krakow, against other universities. The team is composed of around 50 students (management & engineering), and the software team is structured around navigation, robotic arm, and control station subteams.

The navigation, robotic arm as well as the drone team all use ArUco detection. The detection of ArUco tags in different lighting conditions is paramount to the success of those teams. Traditional ArUco approaches (e.g., OpenCV's implementation) rely on thresholding, contour extraction and binary decoding. These methods work very well in ideal conditions but are not robust to partial occlusions such as small leaves or small obstacles on the tags, partial enshadowment or motion blur.

Recent state-of-the-art approaches such as DeepArUco++ demonstrate that robust marker detection under harsh illumination is best achieved through a modular deep pipeline combining (i) bounding-box detection, (ii) corner refinement, and (iii) neural decoding. These methods outperform classical OpenCV ArUco in terms of robustness, while remaining compatible with real-time robotics deployment. The project will build upon these ideas, including synthetic dataset generation with realistic lighting augmentation and systematic evaluation using precision–recall metrics, corner localization error, and decoding accuracy.

Objectives

The main objective is to design and implement a complete **Deep Learning ArUco detection pipeline** suitable for deployment on a Martian-like rover field test.

The student will:

1. Perform a literature review
2. Create a dataset of ArUco markers under varied lighting, occlusion and motion blur conditions
3. Implement YOLOv8 (or similar) based marker detection trained via synthetic and/or real data
4. Implement Deep Corner Refinement
5. Implement neural decoding + Hamming-distance filtering
6. Evaluate robustness and validate in real rover field experiments.

Deliverables

1. **Baseline and failure analysis**
 - Benchmark classical OpenCV ArUco detection under challenging conditions
 - Document failure modes: shadows, occlusions, blur, low-light...
2. **Dataset and augmentation pipeline**
 - Curated dataset of ArUco tags under varied lighting and occlusion
 - Synthetic data generation
 - train/validation/test split
3. **Deep marker detection module**
 - YOLOv8-based (or similar) detector trained for robust marker localization
4. **Corner refinement and decoding module**

- Deep corner refinement
- Marker decoding with Hamming-distance filtering

5. **Integration and real-time prototype**

- Real-time inference on live 1080p USB camera stream

6. **Experimental evaluation report**

- Precision–Recall curves
- Corner localization error (pixels)
- Decoding accuracy vs lighting/occlusion level
- Throughput (FPS) and latency measurements

7. **Report + code release**

- Full methodology, results, and deployment recommendations
- Clean repository with training + inference instructions

Requirements

EPFL Xplore is looking for a motivated student interested in applying deep learning to real robotics perception challenges.

- Strong programming skills in Python (required), C++ is a plus
- Familiarity with Linux development environments
- Good knowledge of classical Computer Vision
- Interest in deep learning for detection tasks (PyTorch, TensorFlow...)
- Ability to run experiments rigorously and report quantitative results
- Comfortable working with robotics middleware (ROS2 preferred)
- **Bonus:**
 - Experience with YOLO-style detectors
 - Previous robotics perception projects

Hardware Recommendations

1. **Camera:** 1080p USB camera
2. **Training workstation:** NVIDIA GPU recommended
3. Printed ArUco tag set for controlled indoor/outdoor experiments

Software Stack Recommendations

(not exhaustive)

1. OpenCV ArUco module for baseline comparison
2. Deep Learning framework:
 - PyTorch (preferred)
3. Ultralytics YOLOv8 for marker detection
4. Data augmentation tools
5. Export + deployment tools (optional):
 - ONNX, TensorRT for embedded acceleration

Resources

- DeepAruco Implementation: https://doi.org/10.1007/978-3-031-36616-1_16
- DeepAruco++ Implementation : <https://arxiv.org/abs/2411.05552>, <https://github.com/AVAuco/deeparuco/>
- ArUco Tag Detection with OpenCV: https://docs.opencv.org/3.4/d9/d6a/group__aruco.html
- Speeding Up ArUco Detection <https://doi.org/10.1016/j.imavis.2018.05.004>
- Deep Autoencoder for low-light image enhancement <https://www.sciencedirect.com/science/article/pii/S003132031630125X?via%3Dihub>